

METROPOLITAN UNIVERSITY, BANGLADESH

Department of Computer Science and Engineering

Compiler Construction Lab

Project Manual

Item	Detail
Project Title	Design and Implement a Mini Programming Language Compiler using Flex and Bison
Project Weight	40% of the Lab Course
Project Submission Deadline	31 July (With no extensions)

NOTICE TO STUDENTS

This project completely replaces the Lab Final Examination. Your final lab grade for this component will be determined entirely through project implementation, documentation, demonstration, presentation, and individual viva. There is no separate written or practical lab final for this course.

1. Introduction and Purpose

The Compiler Construction Laboratory course concludes with a single, comprehensive group project rather than a traditional lab final examination. This project is designed to consolidate everything you have learned throughout the semester: lexical analysis, syntax analysis, semantic analysis, and intermediate code generation, into one working software system.

Throughout the term, you have studied these phases largely in isolation. This project requires you to integrate them into a single coherent pipeline, exactly as a real compiler front-end is structured. Doing so will give you a far deeper understanding of compiler design than any individual lab exercise could provide on its own.

Evaluation will be based on five components:

Component	Description
Project Implementation	The working compiler front-end itself
Documentation	README, code comments, project report
Demonstration	Live execution of the compiler on test programs
Presentation	Group walkthrough of design and architecture
Individual Viva	One-on-one questioning to verify individual understanding

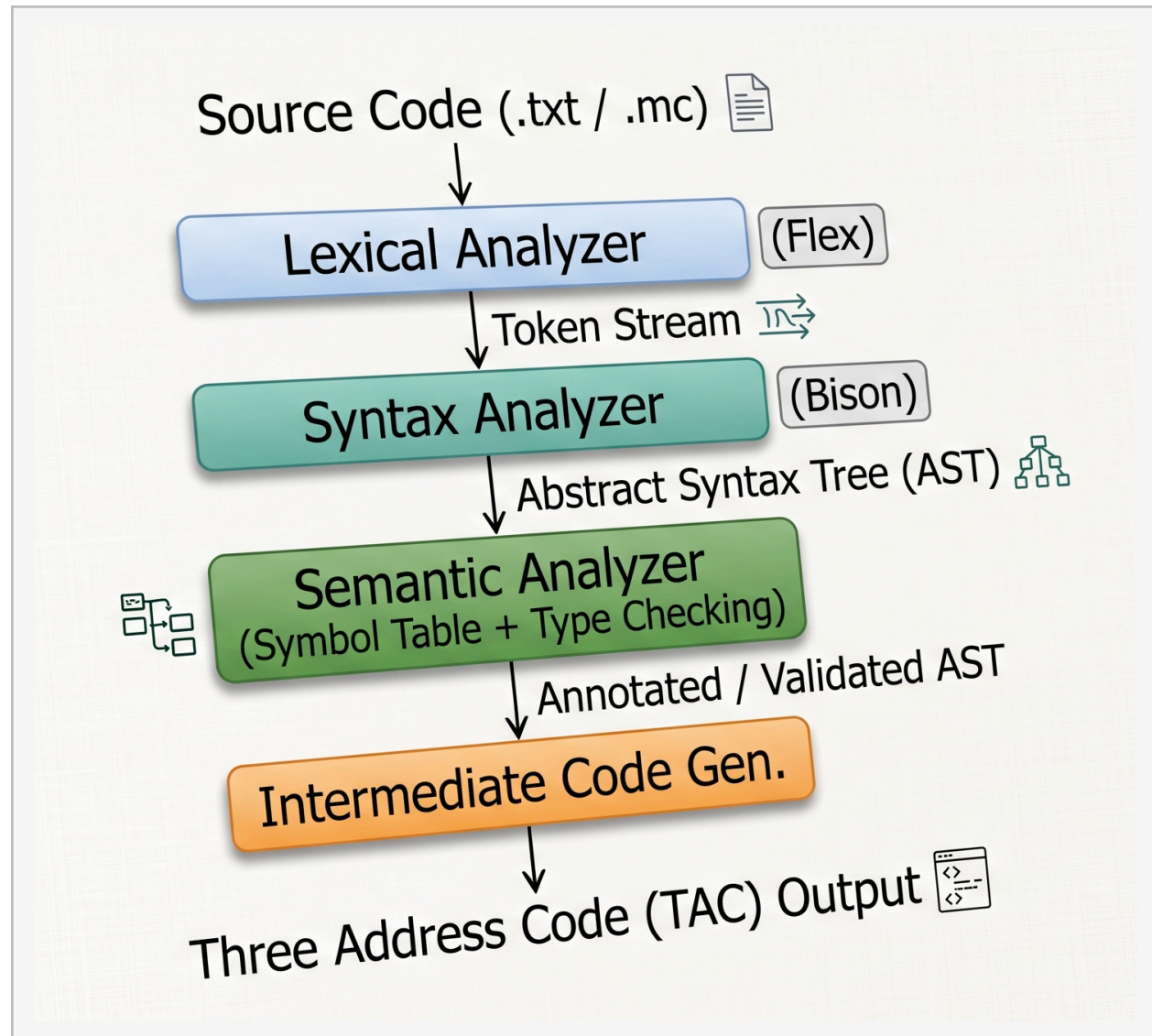
Note

Marks are distributed across all five components. A working compiler with poor documentation or a polished report with no working code will be penalized.

2. Project Overview

Each group will design and implement a compiler front-end with intermediate code generation for a custom programming language that will be defined later in this manual.

The compiler pipeline you must build is illustrated below:



Note

The goal of this project is not to build a production-grade compiler. The goal is to demonstrate, through working code, how the phases of a compiler communicate with and depend on one another. A compiler that correctly rejects invalid programs with clear error messages is just as valuable, academically, as one that correctly compiles valid programs.

3. Important Design Decision

All groups will implement a compiler for the same language, as specified in Section 5 of this manual.

You are not permitted to design your own language, extend the grammar arbitrarily, or substitute a different language of your choosing for the core requirements.

Why a fixed language?

Using one instructor-defined specification for every group ensures that all teams are evaluated on a level playing field. It allows direct, fair comparison of design decisions, code quality, and depth of understanding, rather than comparing projects of varying difficulty. Optional bonus features (Section 14) are where you may differentiate your project.

4. Project Scope (Required Modules)

Your compiler must contain the following six modules. Each is described in detail below.

4.1 Lexical Analyzer (Flex/Lex)

The lexical analyzer is the first phase of your compiler. It reads raw source code character-by-character and groups characters into tokens – the smallest meaningful units of the language.

Your lexer must correctly recognize:

Category	Examples
Keywords	int, float, bool, if, else, while, print, true, false
Identifiers	Variable and identifier names following standard naming rules
Constants	Integer literals, floating-point literals, boolean literals
Operators	Arithmetic, relational, logical operators (see Section 5)
Delimiters	{ } () ;
Comments	Single-line and/or block comments (must be discarded, not tokenized)
Whitespace	Spaces, tabs, newlines (must be discarded)
Invalid Tokens	Any character sequence that does not match a valid token must be reported as a lexical error with line number

Tip

Use Flex's longest-match rule to your advantage; always define keyword patterns before the generic identifier pattern, or use a symbol table lookup to distinguish keywords from identifiers.

4.2 Syntax Analyzer (Bison/Yacc)

The syntax analyzer consumes the token stream produced by the lexer and verifies that it conforms to the Context-Free Grammar (CFG) of the language.

Requirements:

- Define a complete, unambiguous CFG for the language in Section 5.
- Implement the grammar using Bison.
- Detect and report syntax errors with line numbers wherever possible.
- Implement at least basic error recovery (e.g., using Bison's error token) so that a single syntax error does not necessarily halt the entire compilation immediately without any diagnostic output.

4.3 Abstract Syntax Tree (AST)

As parsing succeeds, your compiler must construct an Abstract Syntax Tree — a hierarchical representation of the program's structure with irrelevant syntactic details (such as parentheses and semicolons) removed.

- Each node should represent a meaningful language construct (e.g., IfNode, BinaryOpNode, AssignNode).
- The AST is the data structure passed to both the semantic analyzer and the code generator.
- You must be able to print or visualize the AST in some readable form (text-based indentation is sufficient; Graphviz visualization is a bonus feature — see Section 14).

4.4 Symbol Table

The symbol table tracks every declared identifier and its associated information.

Each entry should record, at minimum:

Field	Purpose
Name	Identifier name
Type	int, float, or bool
Scope	Which block/scope the identifier belongs to

Field	Purpose
Line Declared	For error reporting

Your symbol table must support nested scopes, since the language supports nested blocks (Section 5). A variable declared inside an if or while block must not be visible outside that block.

4.5 Semantic Analyzer

The semantic analyzer walks the AST and validates rules that the grammar alone cannot enforce. It must detect and report, at minimum:

Error Type	Example
Undeclared variable use	Using x before declaring it
Redeclaration	Declaring int x twice in the same scope
Scope violation	Using a variable outside the block in which it was declared
Type mismatch	bool b = 5 + 3.2;
Invalid assignment	Assigning a bool expression to an int variable, or similar
Invalid expressions	e.g., applying logical operators to numeric operands where not permitted

Each detected error should produce a clear, human-readable message that includes (where possible) the line number and a description of the problem.

4.6 Intermediate Code Generation

The final required phase generates Three Address Code (TAC) — a low-level, linear intermediate representation in which each instruction contains at most three operands (typically one operator and up to two operands, with the result stored in a temporary or named variable).

Example — Source code:

```
int a;
int b;
int c;
a = 5;
b = 10;
c = a + b * 2;
```

Generated TAC (illustrative):

```
a = 5
b = 10
t1 = b * 2
t2 = a + t1
c = t2
```

Your generator must correctly handle:

- Arithmetic expressions (respecting operator precedence)
- Relational and logical expressions
- Control flow (if, if-else, while) using labels and conditional/unconditional jumps
- print statements (e.g., print a)

Note

TAC is the final required output of your compiler. You are not required to translate TAC into machine code, assembly, or any executable form (see Section 6).

5. Custom Language Specification

All groups must implement a compiler for the following language. Refer to this section as the authoritative specification: it supersedes any ambiguity elsewhere in this manual.

5.1 Data Types

Type	Description
int	Signed integer
float	Floating-point number
bool	Boolean (true/false)

5.2 Statements Supported

- Variable declaration (e.g., int x;)
- Assignment (e.g., x = 5;)
- Arithmetic expressions
- Relational expressions
- Logical expressions
- if statement

- if-else statement
- while loop
- print statement
- Nested blocks ({ ... }), with proper scoping

5.3 Operators

Category	Operators
Arithmetic	+ - * / %
Relational	< > <= >= == !=
Logical	&& !

5.4 Other Lexical Elements

- Braces: { }
- Parentheses: ()
- Semicolons: ; (statement terminators)
- Identifiers: must start with a letter or underscore, followed by letters, digits, or underscores
- Integer literals (e.g., 42)
- Floating-point literals (e.g., 3.14)
- Boolean literals: true, false

5.5 Sample Program

```
int x;
int y;
bool flag;

x = 10;
y = 0;
flag = true;

while (x > 0) {
    y = y + x;
    x = x - 1;
}

if (flag == true) {
    print y;
} else {
    print x;
}
```


Reminder

You must design and document a complete, formal CFG for this language in your project report (see Section 12), in addition to implementing it.

6. What Is Not Required

To keep the project scope appropriate for a one-semester lab course, the following are explicitly out of scope. Do not spend time on these unless attempting them purely as a personal bonus exploration (which will not be separately graded):

- Machine code generation
- Assembly code generation
- Register allocation
- Linking
- Code optimization (beyond optional bonus items in Section 14)
- Executable/binary generation
- Instruction scheduling
- Any compiler backend targeting real hardware

Warning

Do not let scope creep distract you from delivering a complete, correct front-end. A compiler that fully and correctly implements Sections 4 and 5 will score higher than an incomplete attempt at backend code generation.

7. Recommended Software Stack

Tool	Purpose
Linux (Ubuntu/Debian recommended)	Development environment
Flex	Lexical analyzer generator
Bison	Parser generator
GCC	Compiling your generated C code
Makefile	Build automation
Git	Version control
GitHub	Repository hosting and collaboration
Markdown	Documentation formatting

8. Project Directory Structure

Your repository must follow a clean, professional structure. A suggested layout is shown below:

```

project-root/
├── docs/
├── src/
│   ├── lexer/
│   ├── parser/
│   ├── ast/
│   ├── semantic/
│   └── symbol_table/
├── tests/
├── examples/
├── Makefile
└── README.md
  
```

Folder / File	Purpose
docs/	Project report, diagrams, grammar specification, and any supporting design documents.
src/	All source code, organized by compiler phase.
src/lexer/	Flex specification file(s) (.l) and generated/related lexer code.
src/parser/	Bison grammar file(s) (.y) and generated/related parser code.
src/ast/	AST node definitions, construction logic, and printing/visualization utilities.
src/semantic/	Semantic analysis logic – type checking, scope checking, error detection.
src/symbol_table/	Symbol table data structure and associated operations (insert, lookup, scope management).
tests/	All test programs (valid and invalid) used to validate your compiler, plus expected outputs.
examples/	A small set of representative sample programs demonstrating key language features.
Makefile	Single command (make) to build the entire project from source.
README.md	Project summary, build instructions, usage instructions, and team information.

Tip

Keep generated files (e.g., `lex.yy.c`, `y.tab.c`) out of version control where practical, or clearly mark them as generated. Use a `.gitignore` file.

9. GitHub Requirement

GitHub usage is mandatory and will be directly assessed.

Requirements:

1. Every group must maintain a single shared GitHub repository for the project.
2. Every team member must contribute commits. A repository where all commits originate from one member, while others contributed no code, will raise concerns during evaluation.
3. Commits must be made regularly throughout the project lifecycle — not as one or two large commits near the deadline.
4. Commit messages should be meaningful and descriptive (e.g., “Add scope handling to symbol table”, not “update” or “fix”).

Warning

The instructor may inspect your repository's commit history as part of evaluation. Repositories showing a single massive last-minute commit, or commits concentrated from only one contributor, may result in deductions, even if the final product works correctly. Commit history is one of the tools used to verify individual contribution and effort distribution within the group.

10. AI Usage Policy

Students are permitted to use AI tools, including ChatGPT, Claude, Gemini, GitHub Copilot, and similar tools, while working on this project.

However, the following conditions apply without exception:

- AI assistance does not reduce the expectation of understanding. Every student is expected to fully understand every line of code submitted under their group's name.
- During the presentation and individual viva, any team member may be asked to explain any part of the implementation — regardless of who originally wrote or generated that portion.
- Failure to explain your own group's implementation will result in a significant deduction of marks, even if the compiler functions correctly and even if the explanation is requested for code generated with AI assistance.

- Evaluation is fundamentally based on demonstrated understanding, not merely on authorship or on whether the final program runs.

In short

Use AI as a tool to help you learn and build faster: not as a substitute for understanding.
If you cannot explain it, you should not submit it.

11. Project Deliverables

By the submission deadline, each group must submit:

Deliverable	Notes
Source Code	Complete, buildable source for all required modules
GitHub Repository Link	Must reflect ongoing contribution from all members
Documentation	README + project report (Section 12)
README.md	Build/run instructions, team member list, feature summary
Project Report	Following the structure in Section 12
Presentation Slides	For the in-class presentation (Section 13)
Sample Input Programs	Covering valid and invalid cases
Sample Output	Output corresponding to each sample input
Screenshots	Of successful builds/runs and of error handling
Compilation Instructions	Exact steps/commands to build the project
Execution Instructions	Exact steps/commands to run the compiler on a test file
Video Demonstration	Optional – a short screen recording is welcomed but not required

12. Project Report Structure

Your written report should follow this chapter structure:

5. Introduction – Project context and motivation
6. Objectives – What the project sets out to demonstrate
7. Language Specification – Full formal description, including the CFG
8. Compiler Architecture – Overall pipeline and module interaction
9. Lexer Design – Token definitions, regular expressions used, design decisions

10. Parser Design — Grammar, precedence/associativity rules, conflict resolution
11. Abstract Syntax Tree — Node structure and construction strategy
12. Semantic Analysis — Rules enforced and how each is detected
13. Symbol Table — Data structure design and scope-handling strategy
14. Intermediate Code — TAC generation strategy, with examples
15. Challenges — Technical difficulties encountered and how they were resolved
16. Testing — Summary of test cases and results
17. Conclusion — Reflection on what was learned
18. References — Any external resources consulted

13. Presentation Guidelines

Each group will deliver a short in-class presentation followed by an individual viva. Your presentation should cover:

- Project overview and language summary
- Compiler architecture (the pipeline diagram from your report)
- Walkthrough of each compiler phase
- Live demonstration of the compiler running on at least one valid and one invalid program
- Demonstration of error handling (lexical, syntax, and semantic errors)
- Demonstration of intermediate code generation output
- Challenges faced during development
- Lessons learned

Note

Live demonstration is strongly preferred over slides showing static screenshots. Be prepared for the instructor to request on-the-spot test inputs.

14. Optional Bonus Features

The following features are not mandatory but may be implemented for bonus credit and to differentiate your project from others within the same fixed language specification:

Feature	Category
Arrays	Language extension
Functions, function calls, parameters, return statement	Language extension

Feature	Category
for loop	Language extension
do-while loop	Language extension
switch-case	Language extension
Unary operators, increment/decrement	Language extension
Constant folding	Optimization
Dead code elimination	Optimization
AST visualization with Graphviz	Tooling
Improved error recovery	Robustness
Graphical user interface (GUI)	Tooling

Reminder

Bonus features are evaluated only after all mandatory requirements (Section 4) are satisfactorily met. Do not pursue bonus features at the expense of core functionality.

15. Testing Requirements

Every group must provide a meaningful set of test programs under tests/, demonstrating:

Test Category	Must Demonstrate
Successful Compilation	At least one non-trivial valid program compiling cleanly through to TAC
Lexical Errors	At least one program containing an invalid token, correctly detected
Syntax Errors	At least one program violating the grammar, correctly detected
Semantic Errors	At least one program for each semantic rule in Section 4.5, correctly detected

Each test program should be paired with its expected/actual output for grading reference.

16. Timeline and Submission Policy

Submission Deadline: 31 July

WARNING: STRICT DEADLINE

This deadline is final. No extensions will be granted under any circumstances. Plan your group's schedule accordingly, and do not assume leniency near the deadline.

Additional policies:

- The submission portal will close automatically after the deadline; submissions will not be accepted through it afterward.
- Late GitHub commits: i.e., commits made after the deadline, will be treated as late work and may incur the same penalties as late submission.
- Groups are strongly advised to commit incrementally throughout the project, not upload all work in a single commit near the deadline. Aside from the academic risk, last-minute large commits are themselves treated as a negative indicator during evaluation (see Section 9).
- Late submissions, where accepted at the instructor's discretion before portal closure, will receive a mark deduction proportional to the delay.

17. General Rules and Academic Integrity

- All submitted work must be original to your group. Plagiarism from other groups, prior semesters, or uncredited external sources is a serious academic violation.
- Any external resource, code snippet, or reference material used must be properly cited in your report.
- AI tools are permitted under the policy in Section 10: understanding remains mandatory regardless of tool use.
- Code should follow professional coding style: consistent indentation, meaningful variable/function names, and modular organization.
- Git commit messages must be meaningful and reflect the actual change made.
- Code should be readable: prioritize clarity over cleverness.
- All required documentation (Section 11) must be complete at submission time.

18. Evaluation Rubric (Indicative)

Criterion	Weight
Lexical Analyzer Correctness	10%
Syntax Analyzer Correctness	15%
AST Construction	10%
Symbol Table Design	10%
Semantic Analysis Correctness	20%
Intermediate Code Generation	15%
Documentation & Report Quality	10%
Presentation & Demonstration	10%
Individual Viva (Understanding)	Pass/Fail gate: may scale any of the above for an individual student

Final Note

This rubric is indicative and may be refined by the instructor before grading. The individual viva component exists specifically to verify that grades reflect each student's own understanding, not merely the group's collective output.

End of Project Manual.

Direct any clarification questions to me before the submission deadline, not after.